SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:50**

Annexure A

Scenario based Questions

Code of Conduct:

I ANISH K/192521288 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 6 – CPU Scheduling Scenario

Scenario:

A large-scale CPU scheduling environment in organization 1 experiences performance and

reliability issues during peak usage. Multiple processes/resources interact simultaneously, causing delays, contention, and reduced throughput.

1. Analyze the root cause of the observed behavior.

2. Determine the most appropriate Operating System technique/algorithm to address the issue.

3. Justify your recommendation with respect to CPU utilization, throughput, response time, or fairness.

4. Design an improved system configuration or execution sequence for the given scenario.

5. Evaluate one limitation or trade-off of your proposed solution. **(10 Marks)**

Question 7:

****Scenario:****

A leading international e-commerce company operates across 70 countries and processes nearly 10 million customer requests every day. The system consists of multiple activities such as user authentication, product search, shopping cart updates, payment processing, inventory management, and shipment tracking. During festive sales, the response time of the application increases significantly due to the large number of simultaneously executing tasks. The Chief Technology Officer (CTO) has formed a task force to investigate how the operating system manages these activities as processes and how process state transitions affect overall system performance. The team is also interested in understanding the role of context switching and Process Control Blocks (PCB) in maintaining efficient execution.

****Questions:****

1. Analyze the various processes involved in the given scenario.
2. Differentiate between a program and a process with suitable examples from the case study.
3. Explain how Process Control Blocks (PCB) are utilized by the operating system.
4. Illustrate the different process states that a payment transaction may undergo.
5. Evaluate the impact of context switching during peak sale periods.
6. Predict the consequences of excessive process creation in the system.

7. Design an efficient process hierarchy for the organization.
8. Justify the need for multiprogramming in this environment.
9. Recommend strategies to improve process management and scalability.
10. Develop a process lifecycle model suitable for the e-commerce platform.

Question 8: Process Scheduling

****Scenario:****

A smart hospital has implemented an automated healthcare management system to handle patient registration, diagnostic services, medical record access, pharmacy management, and emergency response operations. During emergency situations, doctors have reported delays in retrieving patient records because CPU resources are being shared among multiple services. The hospital administration has requested the IT team to investigate different CPU scheduling policies and recommend a suitable scheduling strategy that prioritizes emergency services without starving routine tasks.

****Questions:****

1. Identify the scheduling challenges present in the hospital management system.
2. Compare FCFS, SJF, Priority Scheduling, and Round Robin for this application.
3. Analyze the impact of scheduling policies on emergency patient care.
4. Determine the most appropriate scheduling algorithm for the given scenario.
5. Evaluate the possibility of starvation and suggest remedies.
6. Recommend a suitable time quantum if Round Robin is selected.
7. Predict system performance during a mass casualty incident.
8. Justify your choice of scheduling algorithm using performance metrics.
9. Propose modifications to improve fairness and responsiveness.
10. Design a scheduling framework for the hospital's future expansion plans.

Question 9: Inter-Process Communication and Cooperating Processes

****Scenario:****

A smart city traffic management system integrates traffic signals, CCTV cameras, weather stations, public transportation services, and emergency vehicle tracking systems. These

independent components continuously exchange information to optimize traffic flow and reduce congestion. However, communication failures between processes have occasionally resulted in delayed traffic signal updates and emergency response times. The city administration seeks an improved communication model to ensure uninterrupted coordination among all subsystems.

****Questions:****

1. Identify the cooperating processes involved in the smart city ecosystem.
2. Analyze the communication requirements among these processes.
3. Compare shared memory and message passing techniques for this environment.
4. Recommend the most suitable IPC mechanism and justify your choice.
5. Evaluate the impact of communication failures on public safety.
6. Propose a fault-tolerant IPC architecture for the city.
7. Predict potential challenges as the city expands its infrastructure.
8. Suggest methods to improve synchronization among cooperating processes.
9. Design a secure communication framework for the system.
10. Develop a scalability plan for future smart city implementations.

Question 10: Threads and Multithreading Models

****Scenario:****

A global media company streams live sporting events to millions of viewers worldwide. To improve user experience, the application uses separate threads for video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion. During high-demand events such as the FIFA World Cup final, users occasionally experience video freezing and synchronization issues. The software architects have been asked to investigate whether the current multithreading model is appropriate for handling massive workloads.

****Questions:****

1. Identify the threads involved in the streaming application.
2. Compare user-level and kernel-level threads for this environment.
3. Analyze the suitability of One-to-One, Many-to-One, and Many-to-Many models.

4. Evaluate the consequences of thread blocking during live broadcasts.
5. Recommend an appropriate multithreading model with justification.
6. Predict system behavior under a tenfold increase in user traffic.
7. Suggest techniques to improve thread synchronization.
8. Design a thread management strategy for uninterrupted streaming.
9. Evaluate the role of multithreading in enhancing application performance.
10. Develop a monitoring framework for detecting thread-related issues.

RUBRICS FOR CALCULATION
Scenario based assessment

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

Introduction

A railway ticket booking system experiences heavy traffic during festival seasons because thousands of customers try to book tickets simultaneously. The system executes three types of processes: payment gateway transactions (P1), seat availability checks (P2), and ticket confirmation and printing (P3). Since customers have reported delays in ticket confirmation, an efficient CPU scheduling algorithm is required to improve system performance, reduce waiting time, and ensure important tasks are completed quickly.

1. Most Suitable Scheduling Algorithm

The most suitable scheduling algorithm for this scenario is **Priority Scheduling**.

In Priority Scheduling, each process is assigned a priority value. The CPU always executes the process with the highest priority first. Since payment transactions are the most critical operations, they should receive the highest priority, followed by seat availability checks and ticket confirmation.

Priority Order:

- P1 – Payment Gateway Transactions (Highest Priority)
- P2 – Seat Availability Checks (Medium Priority)
- P3 – Ticket Confirmation and Printing (Lowest Priority)

This approach ensures that the most important processes are executed immediately without unnecessary delays.

2. Justification for Choosing Priority Scheduling

Priority Scheduling improves CPU utilization and reduces customer waiting time in several ways.

First, payment gateway transactions are processed immediately, reducing the possibility of payment failures or transaction timeouts. Customers expect successful payment processing without delay, making this process highly critical.

Second, seat availability checks are completed after payment processing. This allows the system to verify ticket availability efficiently before confirming the booking.

Finally, ticket confirmation and printing are executed after the higher-priority tasks. Although confirmation is important, a short delay is acceptable compared to delaying payment processing.

The scheduling algorithm also minimizes unnecessary CPU idle time because the processor continuously executes the highest-priority ready process. During heavy booking periods, this increases CPU utilization and overall system throughput.

Advantages include:

- Fast execution of critical payment operations.
- Reduced waiting time for important customer requests.
- Better CPU utilization during peak traffic.
- Improved response time for high-priority processes.
- Higher customer satisfaction due to faster transaction completion.

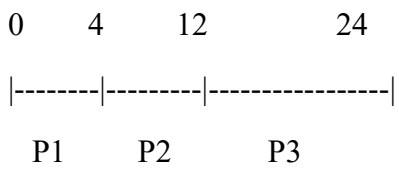
3. Sample Execution Order

Assume the following burst times:

Process	Operation	Priority	Burst Time
P1	Payment Gateway	High	4 ms
P2	Seat Availability Check	Medium	8 ms
P3	Ticket Confirmation & Printing	Low	12 ms

Execution Order:

Time (ms)



Explanation:

- The CPU first executes P1 because it has the highest priority.
- After completing P1, the CPU executes P2.
- Finally, the CPU executes P3.
- This sequence ensures that essential booking operations are completed before less critical tasks.

4. Drawback of Priority Scheduling in Real-Time Systems

One major drawback of Priority Scheduling is **starvation**.

Starvation occurs when low-priority processes continuously wait because higher-priority processes keep arriving. In this ticket booking system, if payment requests are constantly generated during festival seasons, ticket confirmation and printing may experience long delays.

For example:

- New payment requests continue arriving.
- The CPU repeatedly executes high-priority payment processes.
- Ticket confirmation (P3) waits indefinitely.
- Customers experience delayed confirmation even after successful payment.

A common solution to this problem is **Aging**. In the aging technique, the priority of waiting processes gradually increases over time. Eventually, low-priority processes receive higher priority and are executed, preventing starvation and ensuring fairness.

Conclusion

Priority Scheduling is the most suitable CPU scheduling algorithm for a real-time railway ticket booking system because it ensures that critical payment transactions are processed first, reducing response time and improving CPU utilization. It enhances overall system performance during heavy traffic and provides better customer experience. Although starvation is a limitation, implementing the aging technique effectively overcomes this issue, making Priority Scheduling a reliable solution for this application.

Question 2 – Deadlock in a Banking Transaction Server

Introduction

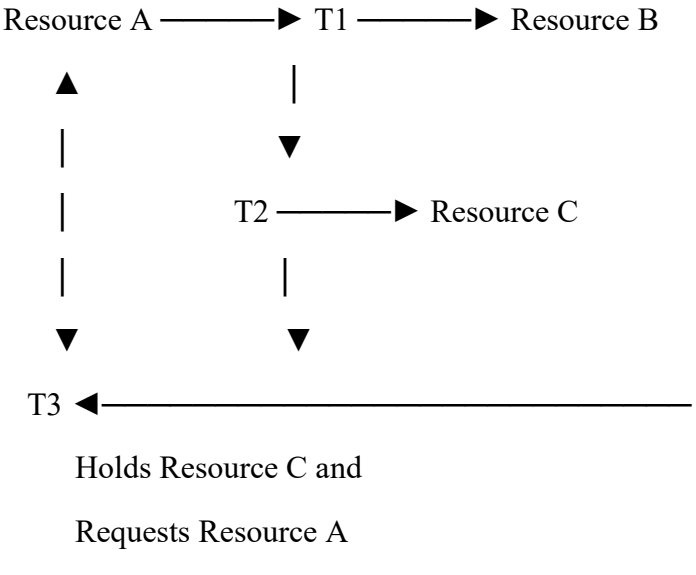
In a banking transaction server, multiple transactions are executed simultaneously to provide fast and efficient customer service. The three operations involved are money transfer (T1), passbook update (T2), and monthly statement generation (T3). These operations require access to shared resources such as the Account Database (A), Customer Information Database (B), and Transaction Logs (C). Improper allocation of these resources can lead to a deadlock, where each transaction waits indefinitely for a resource held by another transaction.

1. Resource Request Graph (Text-Based RAG)

The resource allocation and request sequence is as follows:

- T1 holds **Resource A** and requests **Resource B**.
- T2 holds **Resource B** and requests **Resource C**.
- T3 holds **Resource C** and requests **Resource A**.

Text-Based Resource Allocation Graph:



Another simplified representation:

T1 : Holds A → Waiting for B

T2 : Holds B → Waiting for C

T3 : Holds C → Waiting for A

This forms a circular chain of waiting among the three transactions.

2. Deadlock Identification and Justification

Yes, a **deadlock has occurred**.

A deadlock exists because each transaction is waiting for a resource that is currently held by another transaction. As a result, none of the transactions can continue execution, and the system becomes permanently blocked unless corrective action is taken.

The four necessary conditions for deadlock are satisfied:

Mutual Exclusion

Each database resource can be accessed by only one transaction at a time.

Hold and Wait

Each transaction holds one resource while waiting for another resource.

No Preemption

Resources cannot be forcibly taken away from a transaction until it voluntarily releases them.

Circular Wait

A circular chain exists where:

- T1 waits for B held by T2.
- T2 waits for C held by T3.
- T3 waits for A held by T1.

Since all four conditions are present, the banking server is in a deadlock state.

3. Recommended Technique to Resolve the Deadlock

The recommended technique is **Deadlock Prevention**.

Deadlock Prevention eliminates one of the necessary conditions required for deadlock. In this scenario, the operating system can enforce a predefined order for requesting resources.

For example, every transaction must request resources in the following order:

A → B → C

This means:

- A transaction cannot request Resource B before obtaining Resource A.
- It cannot request Resource C before obtaining Resource B.

By enforcing this fixed order, circular waiting is eliminated, preventing deadlocks from occurring.

Advantages of Deadlock Prevention

- Eliminates deadlocks completely.
- Ensures predictable resource allocation.
- Improves system reliability.
- Suitable for banking applications where data consistency is critical.
- Reduces system downtime caused by blocked transactions.

4. Redesign of the Locking Order

To prevent future deadlocks, the locking mechanism should follow a global resource ordering policy.

Proposed Locking Sequence

Step 1 : Lock Account Database (A)

↓

Step 2 : Lock Customer Information Database (B)

↓

Step 3 : Lock Transaction Logs (C)

↓

Perform Transaction

↓

Release Resources in Reverse Order

(C → B → A)

Using this locking strategy:

- Every transaction follows the same resource acquisition order.

- Circular waiting is removed.
- Resource conflicts are minimized.
- Banking operations execute safely without deadlock.

This approach also improves transaction consistency and simplifies resource management.

Advantages of the Proposed Solution

- Prevents circular waiting.
- Improves transaction reliability.
- Ensures efficient resource utilization.
- Reduces transaction delays.
- Maintains database consistency.
- Enhances overall banking system performance.

Limitations

Although Deadlock Prevention eliminates deadlocks, transactions may wait longer before acquiring all required resources. This can slightly reduce resource utilization during periods of heavy workload. However, in banking systems, maintaining data integrity and preventing transaction failures is more important than achieving maximum resource utilization.

Conclusion

The banking transaction server experiences a deadlock because T1, T2, and T3 form a circular wait while holding different database resources. All four necessary conditions for deadlock are satisfied, causing the system to stop processing transactions. Implementing **Deadlock Prevention** with a fixed resource allocation order (A → B → C) effectively eliminates circular waiting and prevents future deadlocks. This solution improves system reliability, maintains data consistency, and ensures smooth execution of banking operations under heavy workloads.

Question 3 – Mobile OS App Switching Lag

Introduction

A mobile operating system is designed to allow users to switch smoothly between multiple applications while maintaining good performance. In this scenario, users experience delays when switching between apps such as WhatsApp and social media applications, while the background music player also slows down. Although the operating system uses the Round Robin scheduling algorithm with a time quantum of 20 ms, the CPU still shows high idle time despite having 20 background applications. This indicates that the current scheduling strategy is not efficiently utilizing CPU resources and requires improvement.

1. Analysis of High CPU Idle Time

The CPU remains idle even though many applications are active because most of the applications are waiting for input/output (I/O) operations rather than requiring CPU execution.

For example:

- WhatsApp waits for network communication.
- Music player waits for audio buffering.
- Social media applications wait for internet data.
- Background applications remain idle until user interaction occurs.

When these applications are waiting for network, storage, or user input, they enter the **Waiting (Blocked) State**. Since they are not ready for execution, the CPU has no process to execute at certain times, resulting in increased idle time.

Another reason is that having many background applications increases context switching overhead. The CPU spends additional time switching between processes instead of performing useful work, reducing overall efficiency.

Reasons for High CPU Idle Time

- Most applications are waiting for I/O operations.
- Excessive context switching among many background applications.
- Inefficient CPU scheduling for mobile workloads.
- Frequent waiting for network responses.

- Large number of inactive background processes.

2. Effect of the Current Time Quantum

The current **20 ms time quantum** is not optimal for this mobile operating system.

A larger time quantum causes each application to hold the CPU for a longer duration before another application gets a chance to execute. As a result, interactive applications such as WhatsApp and social media experience noticeable delays during app switching.

Although a larger quantum reduces the number of context switches, it also increases response time for interactive applications.

If the time quantum were extremely small, context switching would become excessive and waste CPU time. Therefore, the current quantum is slightly large for a mobile environment where fast user interaction is important.

Effects of a 20 ms Time Quantum

- Increased response time.
- Delayed app switching.
- Slower user interaction.
- Reduced responsiveness.
- Better than very large quantum but not ideal for mobile devices.

3. Recommended Scheduling Method

The recommended solution is **Priority-Based Round Robin Scheduling** with a **reduced time quantum of 10 ms**.

In this approach:

- Foreground applications receive higher priority.
- Background applications receive lower priority.
- Interactive applications are scheduled more frequently.
- CPU time is distributed fairly among processes.

Example Priority Levels:

Application	Priority
WhatsApp	High
Active Social Media App	High
Music Player	Medium
Background Apps	Low

Using a **10 ms time quantum** allows the CPU to switch more quickly between active applications, resulting in faster response and smoother app switching.

Advantages

- Faster response for foreground applications.
- Better user experience.
- Reduced waiting time.
- Improved CPU utilization.
- Balanced fairness among processes.

4. Improvement in Switching Lag and System Throughput

The proposed scheduling method improves performance in several ways.

Foreground applications receive CPU time immediately, making app switching faster and smoother. Since interactive applications are executed more frequently, users experience less delay while opening or switching between apps.

Reducing the time quantum to 10 ms allows more frequent scheduling of ready processes without causing excessive overhead. Background applications continue to execute but do not interfere with important user activities.

The operating system also utilizes CPU resources more efficiently by giving preference to applications currently in use.

Performance Improvements

- Faster application switching.
- Reduced waiting time.
- Better CPU utilization.
- Increased system throughput.

- Improved multitasking performance.
- Smoother user experience.
- Reduced lag during heavy multitasking.

Comparison of Scheduling Methods

Scheduling Algorithm	Suitability	Reason
FCFS	Not Suitable	Interactive apps wait behind long-running processes.
SJF	Partially Suitable	Short processes finish quickly but priorities are ignored.
Round Robin (20 ms)	Moderately Suitable	Fair scheduling but slower response for interactive apps.
Priority-Based Round Robin (10 ms)	**Most Suitable**	Provides quick response to foreground applications while maintaining fairness.

Advantages of the Proposed Solution

- Reduces application switching delay.
- Improves CPU utilization.
- Enhances responsiveness of mobile applications.
- Provides fair CPU allocation.
- Supports efficient multitasking.
- Improves overall user satisfaction.

Limitation

The proposed method increases the number of context switches because of the smaller time quantum. Although this introduces slight scheduling overhead, the improvement in responsiveness and user experience outweighs the additional CPU cost in mobile operating systems.

Conclusion

The mobile operating system experiences switching lag because most background applications spend significant time waiting for I/O operations, resulting in high CPU idle time. The current Round Robin scheduling with a 20 ms time quantum does not provide sufficient responsiveness

for interactive applications. Implementing **Priority-Based Round Robin Scheduling** with a **10 ms time quantum** ensures that foreground applications receive CPU time more quickly, reducing switching lag, improving CPU utilization, increasing system throughput, and providing a smoother user experience.

Question 4 – Deadlock Risk in an Operating System Update

Introduction

Operating system updates involve several modules executing simultaneously to install new features, update system files, and improve security. During this process, different modules compete for shared resources such as Disk I/O, Kernel Locks, and Configuration Locks. If these resources are not allocated properly, the system may enter a deadlock state, causing the update process to stop or fail. Therefore, effective resource allocation and scheduling techniques are essential to ensure successful completion of the operating system update.

1. Identification of Deadlock

Yes, the given sequence can lead to a **deadlock**.

The resource allocation is as follows:

- **M1** holds **R1 (Disk I/O)** and waits for **R2 (Kernel Lock)**.
- **M2** holds **R2 (Kernel Lock)** and waits for **R3 (Configuration Lock)**.
- **M3** holds **R3 (Configuration Lock)** and waits for **R1 (Disk I/O)**.

Each module is waiting for a resource currently held by another module. Since none of the modules can proceed until they obtain the required resource, the update process becomes permanently blocked.

2. Explanation of the Circular Wait Condition

The deadlock occurs because of a **circular wait** among the modules.

The sequence is:

M1 → Holds R1 → Waiting for R2

↓

M2 → Holds R2 → Waiting for R3

↓

M3 → Holds R3 → Waiting for R1

↓

Back to M1

This creates the following cycle:

M1 → R2 → M2 → R3 → M3 → R1 → M1

Since each module waits for a resource held by another module in the cycle, none of them can continue execution.

The four necessary conditions for deadlock are satisfied:

- **Mutual Exclusion:** Resources can be used by only one module at a time.
- **Hold and Wait:** Each module holds one resource while requesting another.
- **No Preemption:** Resources cannot be forcibly taken from a module.
- **Circular Wait:** A closed chain of waiting exists among the modules.

Therefore, the system is in a deadlock-prone state.

3. Banker's Algorithm Style Resource Allocation

To prevent deadlock, the operating system can use **Banker's Algorithm**.

Banker's Algorithm allocates resources only if doing so keeps the system in a **safe state**. Before granting a resource request, the algorithm checks whether all modules can still complete their execution without causing deadlock.

Safe Execution Sequence

The operating system can allocate resources in the following order:

Step 1 : Allocate resources to M1

↓

M1 completes and releases R1

↓

Step 2 : Allocate resources to M2

↓

M2 completes and releases R2

↓

Step 3 : Allocate resources to M3

↓

M3 completes and releases R3

Safe Sequence:

<M1, M2, M3>

Since each module completes before the next module receives additional resources, circular waiting is eliminated.

Advantages of Banker's Algorithm

- Prevents deadlock before it occurs.
- Maintains a safe resource allocation state.
- Improves system reliability.
- Ensures successful completion of OS updates.
- Maximizes efficient resource utilization.

4. Module Prioritization to Improve CPU Utilization

Operating system designers can improve CPU utilization by assigning priorities to update modules based on their importance.

Suggested Priority Order

Module	Function	Priority
M1	Copy Update Files	High
M2	Install Drivers	High
M3	Update Registry	Medium
M4	Validate System Integrity	Low

In this approach:

- Critical modules execute first.
- Independent modules can execute in parallel whenever resources are available.
- Modules waiting for I/O operations allow the CPU to execute other ready modules.
- Resources are released immediately after task completion to reduce waiting time.

This scheduling strategy improves CPU utilization, minimizes idle time, and shortens the overall update duration.

Advantages of the Proposed Solution

- Prevents deadlock during OS updates.
- Improves CPU utilization.
- Reduces waiting time for system resources.
- Ensures safe execution of update modules.
- Increases overall system reliability.
- Reduces the risk of update failures.

Limitation

Banker's Algorithm requires prior knowledge of the maximum resource requirements of each module. In large and complex operating systems, accurately estimating future resource needs can be difficult. Additionally, the algorithm introduces extra computation overhead before granting resource requests.

Conclusion

The given resource allocation sequence can lead to a deadlock because M1, M2, and M3 form a circular wait while holding different resources. By applying **Banker's Algorithm**, the operating system grants resources only when the system remains in a safe state, thereby preventing deadlock. Furthermore, prioritizing critical update modules and releasing resources promptly improves CPU utilization, reduces update time, and ensures a reliable and efficient operating system update process.

Question 5 – Cloud Server CPU Utilization Analysis

Introduction

Cloud computing allows multiple Virtual Machines (VMs) to run on a single physical server, enabling efficient resource sharing and improved performance. In this scenario, the cloud hosting provider observes that VM processes spend **80% of their time waiting for network or disk operations (I/O)** and only **20% of their time using the CPU**. Each server hosts **7 Virtual Machines (VMs)**. To evaluate the server's efficiency, CPU utilization must be calculated and suitable operating system strategies should be recommended to improve system performance.

1. Calculation of CPU Utilization

The CPU utilization in a multiprogramming system is calculated using the formula:

$$\text{CPU Utilization} = 1 - p^n$$

Where:

- **p = I/O wait probability = 80% = 0.8**
- **n = Number of Virtual Machines = 7**

Given Data

- I/O Wait Probability (p) = **0.8**
- Number of VMs (n) = **7**

Calculation

$$\text{CPU Utilization} = 1 - (0.8)^7$$
$$= 1 - 0.2097$$
$$= 0.7903$$
$$\text{CPU Utilization} = 79.03\%$$

Answer

CPU Utilization = 79.03% (approximately 79%)

2. Interpretation of the Result

The calculated CPU utilization is **79.03%**, which indicates that the server is **efficiently utilized but not fully loaded**.

A CPU utilization close to **80%** is generally considered optimal in cloud environments because:

- The CPU remains busy most of the time.
- There is sufficient capacity to handle additional workloads.
- System responsiveness remains good.
- Resources are used efficiently without excessive overload.

Since approximately **21%** of CPU capacity remains unused, the server can still accommodate additional processes or VMs if required.

Therefore, the server is **optimally loaded** rather than underutilized.

3. Effect of Increasing or Decreasing the Number of VMs

The number of Virtual Machines directly affects CPU utilization and system throughput.

Increasing the Number of VMs

When more VMs are added:

- CPU utilization increases.
- Idle CPU time decreases.
- Overall throughput improves because another VM can use the CPU while others wait for I/O operations.
- Better utilization of available hardware resources is achieved.

However, adding too many VMs can lead to:

- CPU contention.
- Memory shortages.
- Increased context switching.
- Reduced overall performance.

Decreasing the Number of VMs

When the number of VMs is reduced:

- CPU utilization decreases.
- CPU remains idle more frequently.
- System throughput decreases.
- Hardware resources are underutilized.

Therefore, maintaining an appropriate number of VMs ensures efficient resource utilization and high system performance.

4. Operating System Strategies to Improve CPU Utilization

The operating system can implement several strategies to improve CPU utilization in the cloud environment.

Strategy 1: Efficient CPU Scheduling

The operating system should use efficient scheduling algorithms such as **Round Robin** or **Priority Scheduling**.

Benefits include:

- Better CPU allocation among VMs.
- Reduced waiting time.
- Improved response time.
- Increased CPU utilization.
- Higher overall throughput.

Strategy 2: Load Balancing

The operating system should distribute Virtual Machines evenly across available physical servers.

Benefits include:

- Prevents server overload.
- Balances CPU usage.
- Reduces response time.
- Improves resource utilization.
- Enhances overall cloud performance.

Advantages of the Proposed Strategies

- Increases CPU utilization.
- Improves overall throughput.
- Reduces CPU idle time.
- Balances workload across servers.

- Enhances resource utilization.
- Improves response time.
- Provides better scalability for cloud services.

Limitation

Although increasing the number of Virtual Machines improves CPU utilization, adding too many VMs can lead to excessive resource contention, increased context switching, and reduced system performance. Therefore, cloud providers must carefully determine the optimal number of VMs based on available CPU, memory, and storage resources.

Conclusion

The CPU utilization of the cloud server is calculated as **79.03%**, indicating that the server is operating efficiently and is close to the optimal utilization level. Increasing the number of Virtual Machines can further improve throughput, provided system resources are not overloaded. By implementing efficient CPU scheduling and load balancing techniques, the operating system can reduce CPU idle time, maximize resource utilization, improve scalability, and ensure better performance in cloud computing environments.

Question 6 – CPU Scheduling Scenario

Introduction

In a large-scale organization, multiple users and applications access the CPU simultaneously during peak working hours. As the number of processes increases, the operating system experiences delays, resource contention, and reduced throughput. Inefficient CPU scheduling may result in longer waiting times, lower CPU utilization, and poor user experience. Therefore, selecting an appropriate CPU scheduling algorithm is essential to improve overall system performance and reliability.

1. Analysis of the Root Cause

The primary reason for the performance degradation is the excessive number of processes competing for CPU resources during peak usage. Since several processes become ready for execution at the same time, the CPU scheduler struggles to allocate processor time efficiently.

The major causes are:

- A large number of processes are waiting in the ready queue.
- High CPU contention among user and system processes.
- Frequent context switching increases scheduling overhead.
- Some long-running processes occupy the CPU for extended periods.
- Poor scheduling decisions increase waiting time and reduce throughput.

As a result, the CPU becomes overloaded, causing slower execution and decreased system responsiveness.

2. Most Appropriate Operating System Technique

The most suitable scheduling algorithm for this scenario is **Priority-Based Round Robin Scheduling**.

This scheduling method combines the advantages of both **Priority Scheduling** and **Round Robin Scheduling**.

In this approach:

- High-priority processes receive CPU access before lower-priority processes.

- Processes with the same priority are executed using Round Robin scheduling.
- Each process receives a fixed time quantum, ensuring fair CPU allocation.
- Critical tasks are completed quickly while maintaining fairness among all processes.

This scheduling technique is widely used in modern operating systems because it provides both responsiveness and fairness.

3. Justification of the Recommendation

Priority-Based Round Robin Scheduling improves several performance parameters.

CPU Utilization

The CPU remains busy because ready processes are continuously scheduled, reducing idle time.

Throughput

More processes are completed within a given time, increasing the number of completed tasks per unit time.

Response Time

High-priority processes receive immediate CPU access, resulting in faster system response.

Fairness

Processes with equal priority receive CPU time equally through the Round Robin mechanism, preventing starvation.

Overall, this scheduling algorithm balances performance, responsiveness, and fairness, making it suitable for large organizational environments.

4. Improved System Configuration and Execution Sequence

The operating system should classify processes into different priority levels before scheduling.

Proposed Priority Levels

Process Type	Priority
System Processes	High
Interactive User Applications	High
Background Services	Medium
Maintenance Tasks	Low

Execution Sequence

High Priority Queue

System Processes

↓

Interactive Applications

↓

Medium Priority Queue

Background Services

↓

Low Priority Queue

Maintenance Tasks

Within each priority queue, the operating system uses **Round Robin Scheduling** with a **time quantum of 10 milliseconds**.

Execution Flow:

Ready Queue

↓

High Priority Processes

↓

Round Robin Scheduling

↓

Medium Priority Processes

↓

Low Priority Processes

↓

Process Completion

This configuration ensures that critical applications execute first while lower-priority processes continue to receive CPU time without starvation.

5. Limitation of the Proposed Solution

Although Priority-Based Round Robin Scheduling improves system performance, it has certain limitations.

- Frequent context switching increases CPU overhead.
- Selecting inappropriate priority levels may delay lower-priority processes.
- Additional scheduling logic slightly increases operating system complexity.
- Very small time quantum values may reduce CPU efficiency due to excessive context switching.

Therefore, the operating system must carefully choose appropriate priority levels and time quantum values to achieve optimal performance.

Advantages of the Proposed Solution

- Improves CPU utilization.
- Increases system throughput.
- Reduces average waiting time.
- Provides faster response for critical processes.
- Ensures fairness among processes with equal priority.
- Prevents starvation through Round Robin scheduling.
- Supports efficient multitasking.
- Enhances overall system reliability.

Conclusion

The performance issues during peak usage are mainly caused by CPU contention, excessive context switching, and inefficient scheduling of multiple processes. Implementing **Priority-Based Round Robin Scheduling** provides an effective solution by prioritizing critical processes while ensuring fair CPU allocation among processes of the same priority. This approach improves CPU utilization, increases throughput, reduces response time, and enhances overall system performance. Although it introduces some scheduling overhead, its advantages significantly outweigh its limitations, making it a suitable scheduling strategy for large-scale organizational environments.

Question 7 – Process Management in an International E-Commerce System

Introduction

A leading international e-commerce company processes nearly 10 million customer requests every day across 70 countries. The operating system manages various activities such as user authentication, product search, shopping cart updates, payment processing, inventory management, and shipment tracking as separate processes. During festive sales, a large number of requests are executed simultaneously, making efficient process management essential for maintaining high system performance, reducing response time, and ensuring customer satisfaction.

1. Analysis of the Various Processes Involved

The operating system executes multiple independent and cooperating processes to handle different business operations.

The major processes are:

- **User Authentication Process** – Verifies customer login credentials and grants secure access.
- **Product Search Process** – Retrieves product information based on customer searches.
- **Shopping Cart Process** – Adds, removes, and updates selected products.
- **Payment Processing Process** – Verifies payment details and completes financial transactions.
- **Inventory Management Process** – Updates product stock after successful purchases.
- **Shipment Tracking Process** – Generates shipment details and provides delivery updates.

These processes execute simultaneously and communicate with one another to provide seamless online shopping services.

2. Difference Between Program and Process

A **program** is a collection of instructions stored on a storage device, whereas a **process** is a program that is currently executing in memory.

Program	Process
Passive entity	Active entity
Stored on disk	Executing in main memory
Does not consume CPU resources until executed	Uses CPU, memory, and I/O resources
Static in nature	Dynamic in nature

Example from the Scenario

- The payment software stored on the server is a **program**.
- When a customer makes an online payment, the payment software starts executing and becomes a **process**.

3. Role of Process Control Block (PCB)

A **Process Control Block (PCB)** is a data structure maintained by the operating system to store all information related to a process.

The PCB contains:

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Memory Allocation Information
- Scheduling Information
- Priority Level
- I/O Status
- Accounting Information

Whenever the CPU switches from one process to another, the operating system saves the current process information into its PCB and restores the next process from its PCB. This enables efficient multitasking and process management.

4. Process States of a Payment Transaction

A payment transaction passes through several process states during execution.

New

↓

Ready

↓

Running

↓

Waiting (Payment Gateway Response)

↓

Ready

↓

Running

↓

Terminated

Explanation

- **New** – Payment request is created.
- **Ready** – Waiting for CPU allocation.
- **Running** – CPU processes the payment.
- **Waiting** – Waiting for bank or payment gateway response.
- **Ready** – Returns to the ready queue after receiving the response.
- **Running** – Completes the transaction.
- **Terminated** – Process finishes successfully.

5. Impact of Context Switching During Peak Sales

Context switching occurs when the CPU changes from one process to another.

During festive sales:

- Millions of customer requests increase the number of active processes.
- Frequent context switching increases CPU overhead.
- CPU spends more time saving and restoring process states.
- Response time increases.
- Overall throughput decreases.

- System performance becomes slower.

Although context switching supports multitasking, excessive switching reduces system efficiency.

6. Consequences of Excessive Process Creation

Creating too many processes can negatively affect system performance.

Possible consequences include:

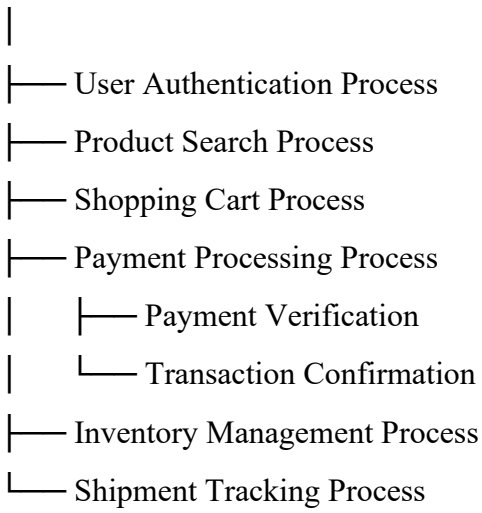
- Increased CPU scheduling overhead.
- Higher memory consumption.
- Longer waiting time.
- Frequent context switching.
- Reduced throughput.
- Poor application responsiveness.
- Increased risk of resource contention.
- Lower overall system efficiency.

Therefore, unnecessary process creation should be avoided.

7. Efficient Process Hierarchy

The operating system can organize processes using a parent-child hierarchy.

E-Commerce Server



This hierarchy simplifies process management and improves coordination between related processes.

8. Need for Multiprogramming

Multiprogramming allows multiple processes to remain in memory simultaneously.

Benefits include:

- Better CPU utilization.
- Reduced CPU idle time.
- Faster execution of customer requests.
- Higher system throughput.
- Improved multitasking.
- Better utilization of memory and I/O devices.

Without multiprogramming, the CPU would remain idle whenever one process waits for I/O operations.

9. Strategies to Improve Process Management and Scalability

The organization can improve process management by implementing the following strategies:

- Use efficient CPU scheduling algorithms.
- Implement load balancing across multiple servers.
- Limit unnecessary process creation.
- Use multithreading for concurrent operations.
- Allocate resources dynamically.
- Monitor system performance continuously.
- Use cloud auto-scaling during festive sales.
- Optimize database access to reduce waiting time.

These strategies improve scalability and ensure smooth operation during heavy workloads.

10. Process Lifecycle Model

The following lifecycle model represents process execution in the e-commerce platform.

New

↓

Ready

↓

Running

↓

Waiting (I/O or Database Access)

↓

Ready

↓

Running

↓

Terminated

This lifecycle enables efficient execution of customer requests while supporting multitasking and resource sharing.

Advantages

- Improves CPU utilization.
- Supports concurrent execution of multiple services.
- Reduces response time.
- Increases system throughput.
- Enhances customer experience.
- Improves scalability during festive sales.
- Ensures efficient resource utilization.
- Supports reliable process management.

Limitation

Managing millions of processes requires significant CPU time and memory resources. During peak sale periods, excessive context switching and process synchronization may reduce system performance. Therefore, efficient scheduling, load balancing, and resource management techniques are essential.

Conclusion

The international e-commerce platform relies on efficient process management to handle millions of customer requests across multiple countries. The operating system manages authentication, product search, shopping cart, payment processing, inventory updates, and shipment tracking as separate processes while maintaining their execution through Process Control Blocks and process state transitions. By implementing multiprogramming, efficient scheduling, and scalable process management strategies, the organization can improve CPU utilization, reduce response time, increase throughput, and deliver reliable services even during high-demand festive sales.

Question 8 – Process Scheduling in a Smart Hospital Management System

Introduction

A smart hospital management system performs several critical operations such as patient registration, diagnostic services, medical record access, pharmacy management, and emergency response. During emergency situations, multiple services compete for CPU resources, resulting in delays in retrieving patient records and slower emergency response. An efficient CPU scheduling algorithm is therefore required to ensure that emergency services receive immediate attention while routine hospital activities continue without starvation.

1. Scheduling Challenges in the Hospital Management System

The hospital management system faces several scheduling challenges due to the simultaneous execution of multiple services.

The major challenges include:

- Emergency requests require immediate CPU allocation.
- Routine hospital services consume CPU resources continuously.
- Large numbers of patient requests increase CPU workload.
- Delays in medical record retrieval affect treatment decisions.
- Multiple processes compete for limited CPU resources.
- Maintaining fairness while prioritizing emergency cases.

These challenges increase response time and reduce the overall efficiency of the hospital system.

2. Comparison of CPU Scheduling Algorithms

Scheduling Algorithm	Advantages	Disadvantages	Suitability
FCFS (First Come First Serve)	Simple to implement	Emergency cases may wait behind routine tasks	Not Suitable
SJF (Shortest Job First)	Reduces average waiting time	Difficult to estimate burst time accurately	Partially Suitable
Priority Scheduling	Executes emergency services first	Low-priority tasks may suffer starvation	Most Suitable

Scheduling Algorithm	Advantages	Disadvantages	Suitability
Round Robin	Fair CPU allocation	Frequent context switching during heavy load	Suitable for equal-priority tasks

Among these algorithms, **Priority Scheduling** is the most appropriate because emergency medical services must always receive immediate CPU access.

3. Impact of Scheduling Policies on Emergency Patient Care

Scheduling policies directly influence the quality of healthcare services.

A poor scheduling policy may result in:

- Delayed emergency treatment.
- Slow access to patient medical records.
- Increased waiting time for doctors.
- Delayed ambulance coordination.
- Reduced quality of patient care.

An efficient scheduling policy ensures:

- Faster emergency response.
- Immediate access to medical records.
- Reduced patient waiting time.
- Improved treatment efficiency.
- Better utilization of hospital resources.

Thus, proper CPU scheduling plays a vital role in saving lives during emergencies.

4. Most Appropriate Scheduling Algorithm

The recommended scheduling algorithm is **Priority Scheduling with Aging**.

The operating system assigns different priority levels based on the importance of each process.

Suggested Priority Levels

Hospital Service	Priority
Emergency Response	Highest

Hospital Service	Priority
Medical Record Access	High
Diagnostic Services	Medium
Pharmacy Management	Medium
Patient Registration	Low

Using this scheduling method:

- Emergency cases receive immediate CPU allocation.
- Critical patient records are accessed quickly.
- Routine services continue without permanent starvation due to the aging technique.

This scheduling method provides both efficiency and fairness.

5. Starvation and Remedies

Priority Scheduling may lead to **starvation**.

Starvation occurs when low-priority processes continuously wait because higher-priority processes keep arriving.

Example:

- Emergency cases continuously enter the system.
- Patient registration processes remain waiting for long periods.

Remedy – Aging

The operating system gradually increases the priority of waiting processes over time.

Benefits of Aging:

- Prevents starvation.
- Ensures fairness.
- Guarantees execution of all processes.
- Maintains balanced CPU utilization.

6. Suitable Time Quantum for Round Robin

If Round Robin scheduling is selected for processes having equal priority, a **time quantum of 10 milliseconds** is recommended.

Advantages of a 10 ms time quantum:

- Fast response for interactive services.
- Reduced waiting time.
- Balanced CPU sharing.
- Smooth multitasking.
- Better responsiveness.

A very small quantum increases context switching, whereas a very large quantum reduces responsiveness.

7. System Performance During a Mass Casualty Incident

During a mass casualty incident, thousands of emergency requests may arrive simultaneously.

Expected system behavior:

- CPU utilization increases significantly.
- Emergency services receive highest priority.
- Routine services experience temporary delays.
- Memory utilization increases.
- Context switching becomes more frequent.
- Overall system throughput remains stable with efficient scheduling.

If proper scheduling is not implemented, the hospital system may become overloaded, affecting emergency medical care.

8. Justification Using Performance Metrics

The recommended scheduling algorithm improves several important performance metrics.

CPU Utilization

The CPU remains active by continuously executing ready processes.

Response Time

Emergency services receive CPU access immediately, reducing response time.

Throughput

More hospital tasks are completed within a given period.

Waiting Time

Emergency patients experience minimal waiting time.

Fairness

The aging technique ensures that lower-priority services also receive CPU time.

These improvements make Priority Scheduling highly suitable for healthcare applications.

9. Modifications to Improve Fairness and Responsiveness

The hospital can further improve scheduling by implementing the following modifications:

- Apply Aging to prevent starvation.
- Use multiple priority queues.
- Dynamically adjust process priorities.
- Introduce load balancing across multiple servers.
- Schedule emergency tasks immediately.
- Allocate dedicated CPUs for critical healthcare services.
- Continuously monitor system performance.

These modifications improve both fairness and responsiveness.

10. Scheduling Framework for Future Expansion

The hospital can adopt the following scheduling framework.

Patient Request

↓

Priority Assignment

↓

Emergency Queue (Highest Priority)

↓

Medical Services Queue

↓

Routine Services Queue

↓

Round Robin Scheduling Within Each Queue

↓

CPU Execution

↓

Process Completion

This framework supports future expansion by efficiently handling increasing numbers of patients and hospital services while maintaining excellent response time and resource utilization.

Advantages

- Provides immediate response to emergency patients.
- Improves CPU utilization.
- Reduces average waiting time.
- Prevents starvation using Aging.
- Supports efficient multitasking.
- Increases overall system throughput.
- Enhances patient safety.
- Easily scalable for future hospital expansion.

Limitation

Priority Scheduling requires accurate assignment of priorities to different hospital services. Incorrect priority assignment may delay important processes or increase CPU overhead. Therefore, the operating system must carefully manage priorities to maintain both fairness and efficiency.

Conclusion

The smart hospital management system requires an efficient CPU scheduling strategy to provide timely medical services during both normal and emergency situations. **Priority Scheduling with Aging** is the most suitable solution because it gives immediate CPU access to emergency services while ensuring fairness for routine tasks. This approach improves CPU utilization, reduces response time, increases throughput, prevents starvation, and supports the future scalability of the hospital management system.

Question 9 – Inter-Process Communication (IPC) and Cooperating Processes in a Smart City Traffic Management System

Introduction

A smart city traffic management system integrates multiple services such as traffic signals, CCTV cameras, weather stations, public transportation, and emergency vehicle tracking. These independent components work together by continuously exchanging information to improve traffic flow, reduce congestion, and ensure public safety. Efficient **Inter-Process Communication (IPC)** is essential for coordinating these processes and maintaining uninterrupted system performance. An effective IPC mechanism ensures reliable communication, synchronization, scalability, and fault tolerance in the smart city environment

1. Cooperating Processes in the Smart City Ecosystem

The traffic management system consists of several cooperating processes that work together to achieve efficient traffic control.

The major cooperating processes include:

- **Traffic Signal Control Process** – Controls traffic lights based on traffic conditions.
- **CCTV Monitoring Process** – Captures and analyzes live traffic footage.
- **Weather Monitoring Process** – Provides weather updates affecting road conditions.
- **Public Transportation Process** – Monitors buses, trains, and public vehicles.
- **Emergency Vehicle Tracking Process** – Detects ambulances, police, and fire vehicles.
- **Traffic Data Analysis Process** – Analyzes traffic density and congestion patterns.
- **Central Traffic Control Server** – Coordinates communication among all subsystems.

These processes cooperate continuously to improve transportation efficiency and road safety.

2. Communication Requirements Among the Processes

The cooperating processes must exchange information quickly and accurately.

The communication requirements include:

- Real-time traffic updates.
- Live CCTV video transmission.
- Weather alerts for traffic management.
- Emergency vehicle location updates.
- Public transport scheduling information.

- Traffic congestion notifications.
- Synchronization of traffic signal timing.
- Secure communication between all system components.

Reliable communication enables the operating system to coordinate multiple processes efficiently.

3. Comparison of Shared Memory and Message Passing

Feature	Shared Memory	Message Passing
Communication Speed	Very Fast	Moderate
Data Sharing	Shared memory area	Messages exchanged between processes
Synchronization	Requires synchronization mechanisms	Built-in synchronization
Suitable for Distributed Systems	Limited	Highly Suitable
Security	Lower	Higher
Scalability	Moderate	High

Since the smart city infrastructure is geographically distributed, **Message Passing** is more suitable than Shared Memory.

4. Recommended IPC Mechanism

The recommended IPC mechanism is **Message Passing**.

In this approach, each subsystem communicates by sending and receiving messages through secure communication channels.

Advantages include:

- Supports distributed systems.
- Reliable communication between distant processes.
- Better security.
- Easier synchronization.
- Fault isolation.
- Improved scalability.

- Efficient communication among heterogeneous devices.

Therefore, Message Passing is the most appropriate IPC mechanism for the smart city traffic management system.

5. Impact of Communication Failures on Public Safety

Communication failures can seriously affect public safety.

Possible consequences include:

- Delayed traffic signal updates.
- Increased traffic congestion.
- Delayed ambulance movement.
- Slower police and fire service response.
- Increased accident risk.
- Incorrect traffic routing.
- Poor coordination among city services.

Reliable IPC is therefore essential for protecting human lives and ensuring efficient emergency response.

6. Fault-Tolerant IPC Architecture

A fault-tolerant IPC architecture ensures uninterrupted communication even if one component fails.

Proposed Architecture

Traffic Signals

↓

Local Traffic Controller

↓

Message Queue

↓

Central Traffic Server

↓

Backup Server

↓

Emergency Response System

Features of the architecture:

- Redundant communication paths.
- Automatic failover to backup servers.
- Reliable message delivery.
- Continuous monitoring of communication links.
- Error detection and recovery mechanisms.

This architecture minimizes communication failures and improves system reliability.

7. Challenges During Future Expansion

As the smart city grows, the traffic management system may face several challenges.

These include:

- Increased number of connected devices.
- Higher communication traffic.
- Greater network congestion.
- Increased synchronization complexity.
- Larger storage requirements.
- Higher processing overhead.
- Increased cybersecurity threats.
- Greater maintenance complexity.

Proper planning and scalable IPC mechanisms are required to overcome these challenges.

8. Methods to Improve Synchronization

Synchronization ensures that cooperating processes work together correctly without data inconsistency.

The operating system can improve synchronization using:

- Semaphores.
- Mutex Locks.

- Monitors.
- Message Queues.
- Event Notifications.
- Time Synchronization Protocols.
- Distributed Coordination Services.

These mechanisms prevent race conditions and ensure accurate communication among processes.

9. Secure Communication Framework

The smart city should implement a secure communication framework to protect sensitive traffic information.

Framework

Traffic Device

↓

Authentication

↓

Encrypted Communication

↓

Message Passing

↓

Central Traffic Server

↓

Access Control

↓

Authorized User

Security measures include:

- User authentication.
- Data encryption.
- Digital certificates.

- Secure communication protocols.
- Access control mechanisms.
- Continuous security monitoring.
- Intrusion detection systems.

This framework protects the system from unauthorized access and cyberattacks.

10. Scalability Plan for Future Smart City Implementations

To support future expansion, the operating system should adopt the following scalability strategies:

- Deploy cloud-based infrastructure.
- Use distributed processing.
- Implement load balancing.
- Increase network bandwidth.
- Add additional traffic control servers.
- Introduce edge computing devices.
- Upgrade communication protocols.
- Continuously monitor system performance.

These strategies ensure that the system can efficiently manage increasing numbers of devices and traffic requests.

Advantages

- Enables real-time communication.
- Improves traffic management efficiency.
- Supports distributed processing.
- Enhances public safety.
- Reduces traffic congestion.
- Improves emergency response.
- Provides better scalability.
- Increases system reliability.

Limitation

Message Passing introduces communication overhead due to message transmission and network latency. In large-scale distributed systems, network failures or heavy traffic may temporarily delay message delivery. However, implementing fault-tolerant communication and backup servers significantly reduces these issues.

Conclusion

The smart city traffic management system depends on efficient cooperation among multiple processes to ensure smooth transportation and public safety. **Message Passing** is the most suitable Inter-Process Communication mechanism because it provides secure, reliable, and scalable communication between geographically distributed subsystems. By implementing fault-tolerant architecture, synchronization techniques, secure communication, and scalable infrastructure, the operating system can improve traffic flow, reduce congestion, enhance emergency response, and support future smart city expansion.

Question 10 – Threads and Multithreading Models in a Global Media Streaming Application

Introduction

A global media streaming company delivers live sporting events to millions of viewers around the world. To provide smooth and uninterrupted streaming, the application performs several tasks simultaneously, including video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion. These tasks are executed using multiple threads. During high-demand events such as the FIFA World Cup Final, users experience video freezing and synchronization problems. Therefore, an efficient multithreading model is essential to improve performance, scalability, and user experience.

1. Threads Involved in the Streaming Application

The streaming application uses multiple threads, each responsible for a specific task.

The main threads are:

- **Video Decoding Thread** – Decodes video frames for playback.
- **Audio Synchronization Thread** – Synchronizes audio with the video.
- **Subtitle Generation Thread** – Displays subtitles during streaming.
- **Buffering Thread** – Downloads and stores video data before playback.
- **Advertisement Thread** – Inserts advertisements at scheduled intervals.
- **Network Communication Thread** – Receives streaming data from the server.
- **User Interface Thread** – Handles user interactions such as play, pause, and volume control.

These threads execute concurrently to provide smooth streaming.

2. Comparison of User-Level Threads and Kernel-Level Threads

Feature	User-Level Threads	Kernel-Level Threads
Management	Managed by user library	Managed by Operating System
Speed	Faster creation and switching	Slightly slower
Parallel Execution	Limited	Supports true parallel execution
Blocking	One blocked thread blocks all threads	Only the blocked thread is affected
Multicore Support	Limited	Fully supported

For a large-scale streaming application, **Kernel-Level Threads** are more suitable because they support multicore processors and prevent the entire application from stopping when one thread blocks.

3. Suitability of Multithreading Models

Many-to-One Model

- Many user threads are mapped to one kernel thread.
- Simple to implement.
- Does not support parallel execution.
- If one thread blocks, the entire process stops.
- Not suitable for live streaming.

One-to-One Model

- Each user thread is mapped to one kernel thread.
- Supports true parallelism.
- Better responsiveness.
- Efficient use of multicore processors.
- Suitable for streaming applications.

Many-to-Many Model

- Many user threads are mapped to multiple kernel threads.
- Provides excellent scalability.
- Supports high concurrency.
- Efficient resource utilization.
- Most suitable for handling millions of users.

Among these models, **Many-to-Many** provides the best balance between performance and scalability

4. Consequences of Thread Blocking During Live Broadcasts

Thread blocking can significantly affect streaming quality.

Possible consequences include:

- Video freezing.

- Audio and video synchronization issues.
- Delayed subtitle display.
- Increased buffering time.
- Advertisement interruption.
- Poor user experience.
- Increased server workload.
- Loss of viewer satisfaction.

Therefore, blocking should be minimized to maintain uninterrupted streaming.

5. Recommended Multithreading Model

The recommended multithreading model is the **Many-to-Many Model**.

Reasons:

- Supports a large number of concurrent users.
- Efficiently utilizes multicore processors.
- Prevents complete application blocking.
- Improves scalability.
- Reduces resource consumption.
- Enhances overall system performance.
- Provides better fault tolerance.

This model is widely used in high-performance server applications.

6. System Behavior Under a Tenfold Increase in User Traffic

If user traffic increases ten times during a major sporting event, the system will experience:

- Increased CPU utilization.
- Higher memory consumption.
- Greater network bandwidth usage.
- Increased thread creation.
- Higher server workload.
- Possible buffering delays if resources are insufficient.

If the operating system uses an efficient multithreading model with load balancing, the streaming service can continue operating smoothly despite the increased workload.

7. Techniques to Improve Thread Synchronization

The operating system can improve synchronization using the following techniques:

- Mutex Locks.
- Semaphores.
- Condition Variables.
- Monitors.
- Read-Write Locks.
- Atomic Operations.
- Thread Pools.
- Synchronization Barriers.

These techniques prevent race conditions and ensure that shared resources are accessed safely.

8. Thread Management Strategy

The streaming application should implement the following thread management strategy.

User Request

↓

Thread Pool

↓

Video Thread

Audio Thread

Subtitle Thread

Buffering Thread

Advertisement Thread

↓

Synchronization Manager

↓

Media Output

This strategy reduces thread creation overhead and improves resource utilization by reusing existing threads.

9. Role of Multithreading in Enhancing Application Performance

Multithreading significantly improves application performance by allowing multiple tasks to execute simultaneously.

Benefits include:

- Faster video processing.
- Smooth audio-video synchronization.
- Efficient buffering.
- Improved CPU utilization.
- Better responsiveness.
- Reduced waiting time.
- Higher throughput.
- Enhanced user experience.
- Better utilization of multicore processors.

These improvements enable the streaming platform to support millions of viewers efficiently.

10. Monitoring Framework for Detecting Thread-Related Issues

A monitoring framework helps identify and resolve thread-related problems before they affect users.

Proposed Framework

Streaming Server

↓

Thread Monitor

↓

CPU Usage Monitor

↓

Memory Usage Monitor

↓

Deadlock Detection

↓

Performance Analyzer

↓

Alert & Logging System

↓

Administrator Dashboard

The framework continuously monitors:

- CPU utilization.
- Memory usage.
- Thread status.
- Deadlocks.
- Synchronization delays.
- Response time.
- Error logs.
- System performance.

Early detection allows administrators to resolve issues quickly and maintain uninterrupted streaming services.

Advantages

- Supports concurrent execution of multiple tasks.
- Improves CPU utilization.
- Enhances streaming performance.
- Reduces response time.
- Provides better scalability.
- Supports multicore processors.
- Improves user experience.

- Ensures smooth live broadcasting.

Limitation

Multithreading increases system complexity because multiple threads share common resources. Improper synchronization can lead to race conditions, deadlocks, or thread starvation. Additionally, creating too many threads increases memory usage and context-switching overhead. Therefore, efficient thread management and synchronization techniques are necessary for optimal performance.

Conclusion

The global media streaming application relies on multithreading to execute video decoding, audio synchronization, buffering, subtitle generation, and advertisement insertion simultaneously. The **Many-to-Many multithreading model** is the most suitable choice because it provides high scalability, efficient CPU utilization, and better fault tolerance for millions of concurrent users. By implementing proper thread synchronization, thread pooling, continuous monitoring, and efficient resource management, the operating system can deliver smooth, high-quality live streaming even during large-scale events such as the FIFA World Cup Final.